



UNIVERSIDAD
CATÓLICA
BOLIVIANA

DOCUMENTO:

Documento 3 - Documento de Arquitectura de Software — Modelo
C4 (Niveles 1 al 3)

CARRERA:

Ingeniería de Sistemas – SIS

MATERIA:

Ingeniería de Software SIS-213

DOCENTE:

Miguel Ángel Pacheco Arteaga

ALUMNOS:

Pedro Andrez Condorena Apaza
Oziel Rodman Ramos Torrez
Luis Daniel Rojas Cáceres

FECHA:

22 de Septiembre, 2026

ÍNDICE

1.		
1.	Introducción	2
1.1.	Propósito del documento	2
1.2.	Alcance	2
1.3.	Enfoque del modelo C4	3
1.4.	Definiciones y abreviaturas	3
2.	Visión general de la arquitectura	4
2.1.	Entornos de ejecución	4
2.2.	Lógica de dominio compartida	5
2.3.	Stack tecnológico	5
2.4.	Drivers y principios arquitectónicos	6
3.	C1 – Diagrama de contexto	7
3.1.	Diagrama de contexto	7
3.2.	Frontera del sistema	7
3.3.	Personas y actores	8
3.4.	Sistemas externos	8
3.5.	Interpretación del contexto	8
4.	C2 – Diagrama de contenedores	9
4.1.	Diagrama de contenedores	9
4.2.	Contenedores locales	9
4.3.	Contenedores cloud	10
4.4.	Almacenamiento local y central	10
4.5.	Protocolos y comunicación	11
4.6.	Operaciones con y sin conectividad	11
5.	C3 – Diagrama de componentes: Aplicación Desktop	12
5.1.	Diagrama de componentes de la Aplicación Desktop	13
5.2.	Capa de presentación	13
5.3.	Servicios locales y lógica compartida	14
5.4.	Persistencia local y sincronización	14
5.5.	Flujo de una venta sin conexión	14
6.	C3 – Diagrama de componentes: Backend Cloud	15

6.1.	Diagrama de componentes del Backend Cloud	15
6.2.	Punto de entrada y seguridad	15
6.3.	Servicios cloud y sus componentes.....	16
6.4.	Criterio de granularidad de los servicios.....	16
6.5.	Acceso a PostgreSQL por schema	16
7.	Sincronización local–cloud	16
7.1.	Flujo general.....	17
7.2.	Estructura del mensaje Outbox.....	17
7.3.	Sincronización bidireccional y versiones.....	17
7.4.	Idempotencia, conflictos y trazabilidad	18
8.	Registros de decisiones arquitectónicas (ADR)	18
8.1.	ADR-001: Framework del backend.....	18
8.2.	ADR-002: Estrategia de persistencia	19
8.3.	ADR-003: Mecanismo de sincronización.....	19
8.4.	ADR-004: Lógica de dominio compartida entre Desktop y servidor.....	20
8.5.	ADR-005: PostgreSQL central con schemas por dominio.....	21
8.6.	ADR-006: Servicios cloud separados	21
8.7.	ADR-007: SQLite única por puesto	22
8.8.	ADR-008: Sync Service como fachada de sincronización	22
9.	Relación con los requisitos y atributos de calidad.....	23
10.	Conclusiones.....	24

Documento de arquitectura de software — Modelo C4 (niveles 1 al 3)

1. Introducción

El presente documento describe la arquitectura de software del Sistema de Gestión para Farmacéutica mediante el modelo C4 propuesto por Simon Brown. Su finalidad es representar de forma gráfica e interpretativa cómo está organizada la solución, desde su relación con las personas y los sistemas externos hasta la estructura interna del servicio backend.

La arquitectura se construye a partir de los requisitos funcionales y no funcionales consolidados en la Especificación de Requisitos de Software (Documento 2), en particular la operación de funciones críticas sin conexión a Internet, el control de dispositivos autorizados, la trazabilidad de operaciones y la posibilidad de crecer hacia nuevas sucursales.

1.1. Propósito del documento

El propósito de este documento es establecer una visión compartida de la arquitectura para los integrantes del proyecto, de forma que las decisiones técnicas puedan comprenderse, justificarse y verificarse antes y durante el desarrollo.

Además, el documento sirve como referencia para relacionar los requisitos del SRS con los contenedores y componentes que los implementan, y registra formalmente las decisiones arquitectónicas más relevantes mediante fichas ADR con su contexto, alternativas evaluadas, decisión y consecuencias.

1.2. Alcance

El documento cubre los tres primeros niveles del modelo C4: contexto (C1), contenedores (C2) y componentes (C3). El nivel C3 se aplica a dos contenedores: la Aplicación Desktop, que ejecuta localmente un subconjunto de los casos de uso del negocio, y el Backend Cloud, que concentra la seguridad, la sincronización y la información consolidada.

El nivel de código (C4) no forma parte del alcance, ya que su detalle se documentará mediante diagramas de clases y la propia estructura del repositorio durante la etapa de desarrollo.

1.3. Enfoque del modelo C4

El modelo C4 organiza la arquitectura como una serie de mapas con distinto nivel de acercamiento, de manera similar al zoom de un mapa geográfico. Cada nivel responde a una pregunta distinta y está dirigido a una audiencia concreta:

- **C1 – Contexto:** muestra el sistema como una caja única, las personas que lo utilizan y los sistemas externos con los que se comunica. Está dirigido a stakeholders técnicos y no técnicos.
- **C2 – Contenedores:** descompone el sistema en aplicaciones y almacenes de datos ejecutables o desplegables por separado, indicando tecnologías y protocolos de comunicación.
- **C3 – Componentes:** realiza un acercamiento a un contenedor para mostrar sus bloques lógicos internos, sus responsabilidades y sus dependencias.

Los diagramas siguen la notación habitual del modelo: personas en azul oscuro, el sistema y sus contenedores en tonos de azul, los componentes en azul claro y los sistemas externos en gris. Cada relación indica su propósito y, entre corchetes, la tecnología o protocolo utilizado.

1.4. Definiciones y abreviaturas

Término	Definición
C4	Modelo de representación de arquitectura basado en Contexto, Contenedores, Componentes y Código
ADR	Architecture Decision Record: registro formal de una decisión arquitectónica
API	Interfaz de programación de aplicaciones expuesta por el backend
REST	Estilo de comunicación sobre HTTP basado en recursos y representaciones JSON
JWT	JSON Web Token utilizado para autenticar y autorizar solicitudes
MVVM	Patrón Model–View–ViewModel para separar la interfaz de la lógica de presentación
ORM	Mapeador objeto–relacional; en este proyecto, Entity Framework Core
Outbox	Patrón que registra los cambios pendientes en una tabla local para enviarlos de forma confiable
SignalR	Biblioteca de ASP.NET Core para comunicación en tiempo real sobre WebSocket
Factura simulada	Comprobante generado internamente con numeración propia, sin validez fiscal y sin envío al Servicio de Impuestos Nacionales
POS	Punto de venta utilizado por los cajeros
FEFO	Salida prioritaria de los lotes con fecha de vencimiento más próxima

Término	Definición
Local-first	Enfoque en el que el puesto ejecuta y persiste localmente las operaciones críticas y las sincroniza después con la infraestructura central
Schema	Espacio de nombres de PostgreSQL que agrupa las tablas de un dominio dentro de una misma base de datos
Sync Service	Servicio cloud que actúa como punto único de entrada para la sincronización de los puestos
API Gateway	Punto de entrada único que enruta las solicitudes externas hacia los servicios cloud correspondientes

2. Visión general de la arquitectura

La solución adopta una arquitectura local-first. Cada computadora de la farmacia ejecuta una aplicación de escritorio con su propia base de datos SQLite, lo que le permite continuar con las ventas, la caja y las consultas de inventario aunque se interrumpa la conexión a Internet. En la nube, un conjunto de servicios ASP.NET Core concentra la seguridad, la sincronización, las notificaciones y la información consolidada del negocio en una base PostgreSQL central.

La aplicación no es simplemente un cliente que consume una API. Es un cliente local-first que posee capacidad de ejecutar localmente un subconjunto de los casos de uso del negocio y posteriormente sincronizarlos con la infraestructura central.

En consecuencia, PostgreSQL es la fuente consolidada central del negocio, mientras que SQLite constituye la fuente operativa local del puesto durante períodos sin conectividad. SQLite no se considera una caché, porque contiene operaciones reales realizadas sin conexión que todavía no han sido sincronizadas.

2.1. Entornos de ejecución

La arquitectura se organiza en tres bloques que comparten un mismo lenguaje (C#) y una misma lógica de negocio:

- **Lógica compartida:** bibliotecas Pharmacy.Domain y Pharmacy.Application con las reglas de ventas, FEFO, inventario, caja y validaciones de dominio, utilizadas tanto por el entorno local como por el entorno cloud.
- **Entorno local:** aplicación Avalonia Desktop con servicios de aplicación locales, una única base SQLite por puesto, una tabla Outbox y un Sync Worker que envía y recibe cambios.

- **Entorno cloud:** servicios ASP.NET Core desplegados en Docker (Sync, Identity, Notifications y el servicio de negocio) sobre una instancia PostgreSQL administrada organizada mediante schemas por dominio.

2.2. Lógica de dominio compartida

Para evitar duplicar las reglas de negocio entre el puesto y el servidor, las reglas que pueden ejecutarse en ambos entornos se ubican en bibliotecas C# compartidas. Estas bibliotecas no contienen dependencias de SQLite, PostgreSQL, HTTP, SMTP ni de proveedores externos; la persistencia y las integraciones se resuelven mediante interfaces implementadas en cada entorno.

Proyecto	Contenido y responsabilidad
Pharmacy.Domain	Entidades, objetos de valor y reglas de dominio: cálculo de ventas, FEFO, bloqueo de lotes vencidos, reglas de stock y reglas básicas de caja
Pharmacy.Application	Casos de uso, comandos, validaciones con FluentValidation e interfaces de repositorios y servicios que cada entorno implementa
Pharmacy.Desktop	Aplicación Avalonia (Views y ViewModels), servicios locales, persistencia SQLite, Outbox y Sync Worker
Backend	Servicios ASP.NET Core: Sync Service, Identity Service, Notification Service y Business Service (ventas, inventario, caja, compras, proveedores, clientes, recetas y reportes)

2.3. Stack tecnológico

La siguiente tabla resume las tecnologías seleccionadas para cada capa de la solución:

Capa	Tecnología	Uso en el sistema
Frontend / Desktop	C# + Avalonia UI	Aplicación de escritorio multiplataforma para cajeros, administración y gerencia
Patrón de UI	MVVM	Separación entre vistas, ViewModels y modelos para facilitar pruebas y mantenimiento
Lógica compartida	Bibliotecas .NET (Pharmacy.Domain, Pharmacy.Application)	Reglas de negocio ejecutables en el puesto y en el servidor sin duplicar código
Base de datos local	SQLite (una base por puesto)	Fuente operativa local con tablas separadas lógicamente por módulo
Acceso a datos local	Entity Framework Core / Dapper	EF Core para el modelo de dominio y migraciones; Dapper para consultas de lectura intensiva
Backend / servicios cloud	ASP.NET Core Web API (.NET 8)	Servicios Sync, Identity, Notifications y de negocio desplegables por separado
Punto de entrada cloud	API Gateway (YARP sobre ASP.NET Core)	Enrutamiento, terminación TLS y límites de solicitudes hacia los servicios
Base de datos en nube	PostgreSQL administrado con schemas por dominio	Fuente consolidada central con separación por schema y usuarios por servicio

Capa	Tecnología	Uso en el sistema
Sincronización	Outbox Pattern + Sync Worker + Sync Service	Envío confiable, idempotente y bidireccional entre puestos y servidor
Segundo plano	BackgroundService / IHostedService	Sync Worker local, despacho de eventos, notificaciones y alertas de vencimiento
Comunicación	REST (HTTPS + JSON); SignalR opcional	REST para todas las operaciones; WebSocket/SignalR solo para avisos en tiempo real
Autenticación	ASP.NET Core Identity + JWT / refresh tokens	Gestión de usuarios, roles, dispositivos autorizados y renovación segura de tokens
Actualizaciones desktop	Velopack	Distribución e instalación automática de nuevas versiones de la aplicación
Despliegue backend	Docker	Una imagen por servicio para cualquier VPS o proveedor cloud
Infraestructura	PostgreSQL administrado + VPS/cloud	Base de datos gestionada y servidor para los contenedores de los servicios
Logging	Serilog	Registro estructurado con identificador de correlación entre puesto y servicios
Validaciones	FluentValidation	Reglas de validación compartidas en Pharmacy.Application
Testing	xUnit	Pruebas unitarias del dominio compartido y pruebas de integración de servicios y sincronización

2.4. Drivers y principios arquitectónicos

Las decisiones de arquitectura responden a los atributos de calidad y restricciones identificados en el SRS. Los principales drivers son los siguientes:

- **Continuidad operativa:** las funciones críticas de venta, caja y consulta de stock se ejecutan localmente ante una caída de Internet y se sincronizan al restablecerse la conexión.
- **Integridad y trazabilidad:** cada operación se registra de forma atómica con su usuario, dispositivo, fecha y versión, sin pérdida ni duplicación durante la sincronización.
- **Reglas únicas:** las reglas de negocio se escriben una sola vez en la lógica compartida y se ejecutan de igual forma en el puesto y en el servidor.
- **Seguridad:** el acceso se controla por roles y permisos, los cajeros solo operan desde computadoras autorizadas y cada servicio cloud accede únicamente a su schema.
- **Separación con fronteras claras:** Identity, Sync y Notifications se despliegan como servicios independientes, mientras que los dominios con fuerte necesidad transaccional permanecen agrupados.

- **Evolución y costo moderado:** la arquitectura admite nuevas sucursales y la extracción futura de dominios sin incorporar infraestructura innecesaria para una empresa en etapa de apertura.

3. C1 – Diagrama de contexto

El diagrama de contexto representa el nivel de mayor abstracción. El Sistema de Gestión para Farmacéutica se muestra como una única caja, rodeada por las personas que lo utilizan y por los sistemas de terceros con los que intercambia información. En este nivel no se detallan tecnologías internas, sino responsabilidades y relaciones.

3.1. Diagrama de contexto

Figura 1. Diagrama de contexto (C1)
Sistema de Gestión para Farmacéutica

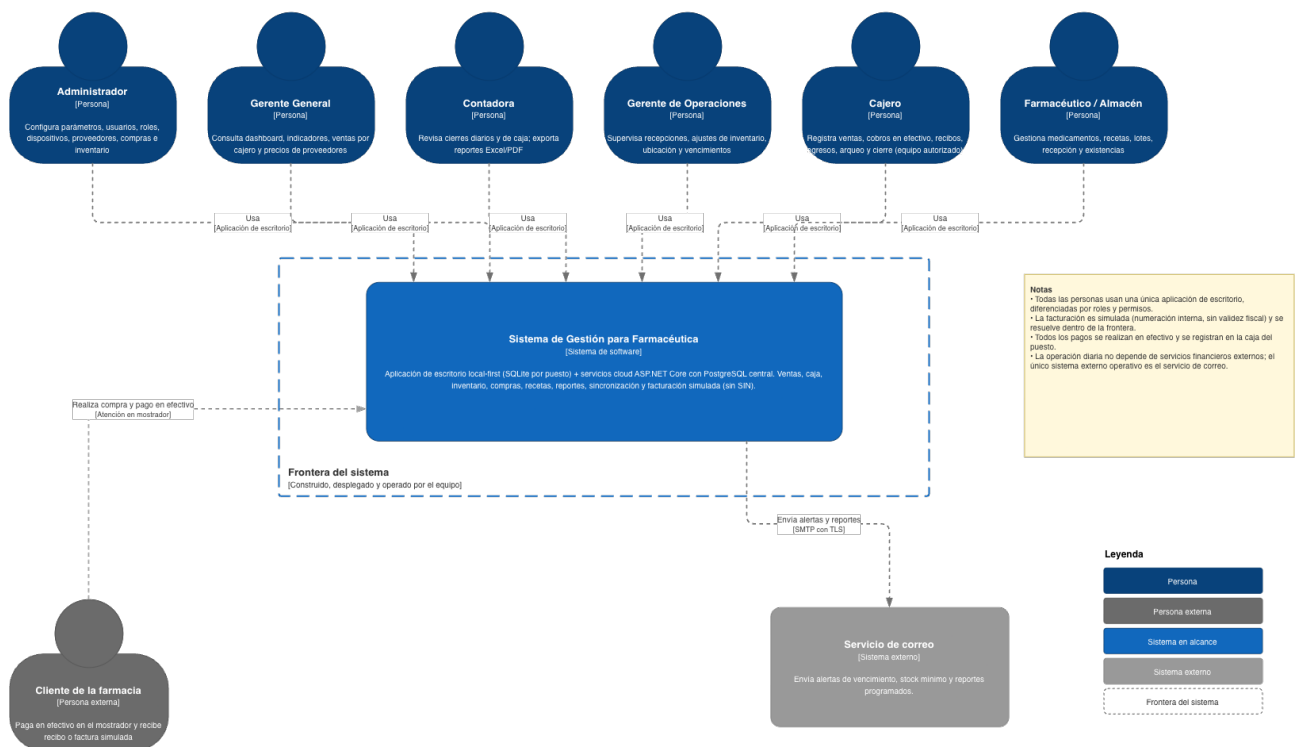


Figura 1. Diagrama de contexto (C1) del Sistema de Gestión para Farmacéutica

3.2. Frontera del sistema

La frontera del sistema, representada con línea discontinua, delimita lo que el equipo del proyecto construye, despliega y opera. Dentro de ella se encuentran la aplicación de escritorio con su base SQLite local, los servicios cloud desarrollados en ASP.NET Core y la base de datos central PostgreSQL. Fuera de ella quedan las plataformas de terceros, cuya disponibilidad, contratos de integración y evolución no dependen del proyecto.

Esta delimitación permite identificar los puntos donde deben aplicarse controles especiales: tiempos de espera, reintentos, validación de respuestas y registro de errores en toda comunicación que cruza la frontera.

3.3. Personas y actores

Persona	Interacción con el sistema	Canal
Administrador	Configura parámetros, usuarios, roles, dispositivos autorizados, proveedores, compras e inventario	Aplicación de escritorio
Gerente General	Consulta el dashboard, indicadores, ventas por cajero y comparación de precios de proveedores	Aplicación de escritorio
Contadora	Revisa cierres diarios y de caja y exporta reportes en Excel o PDF	Aplicación de escritorio
Gerente de Operaciones	Supervisa recepciones de compras, ajustes de inventario, ubicación física y vencimientos	Aplicación de escritorio
Cajero	Registra ventas, cobros en efectivo, recibos, egresos, arqueo y cierre de caja	Aplicación de escritorio en equipo autorizado
Farmacéutico / Almacén	Gestiona medicamentos, recetas, lotes, recepción de productos y existencias	Aplicación de escritorio

3.4. Sistemas externos

Sistema externo	Propósito de la integración	Protocolo
Servicio de correo	Envío de alertas de vencimiento, stock mínimo y reportes programados	SMTP con TLS

3.5. Interpretación del contexto

El diagrama evidencia que todas las personas interactúan con el sistema a través de una única aplicación de escritorio, diferenciándose por roles y permisos y no por aplicaciones distintas. Esto simplifica la distribución del software y concentra el control de acceso en un solo punto.

La facturación es simulada y se resuelve dentro de la frontera del sistema: las facturas y recibos se generan con numeración interna, sin validez fiscal y sin conexión con el Servicio de Impuestos Nacionales (SIN). Por ello, la facturación no depende de ningún sistema externo.

Todos los pagos se realizan en efectivo y se registran en la caja del puesto; el sistema no utiliza pagos electrónicos ni se conecta con entidades bancarias o pasarelas de pago. Por ello, la operación diaria no depende de ningún sistema externo; el único sistema externo es el servicio de correo, utilizado para alertas y reportes.

4. C2 – Diagrama de contenedores

El diagrama de contenedores realiza el primer acercamiento dentro de la frontera del sistema. Un contenedor es una unidad ejecutable o un almacén de datos que se despliega de forma independiente. En este nivel se distinguen los contenedores locales, que se ejecutan en cada puesto de la farmacia, los contenedores cloud, que se ejecutan en el servidor, y los sistemas externos con los que se comunican.

4.1. Diagrama de contenedores

Figura 2. Diagrama de contenedores (C2)
Sistema de Gestión para Farmacéutica

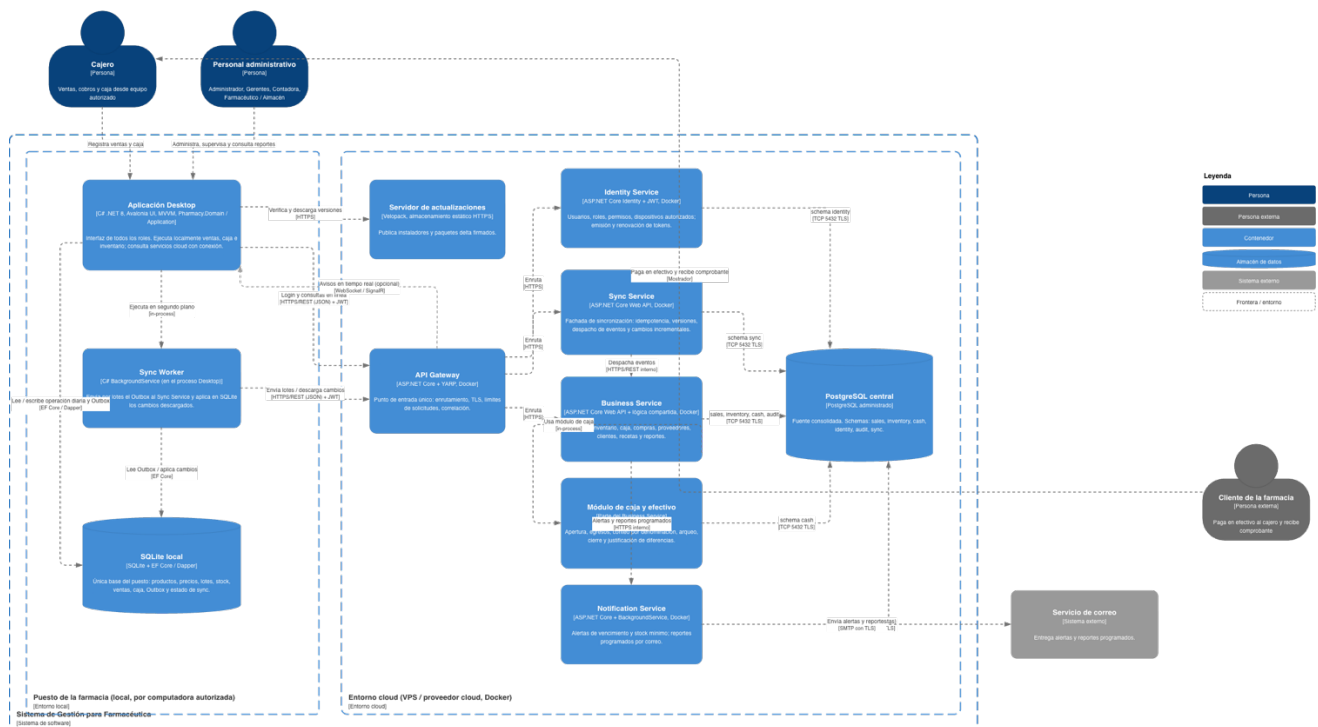


Figura 2. Diagrama de contenedores (C2) del Sistema de Gestión para Farmacéutica

4.2. Contenedores locales

Los contenedores locales se instalan en cada computadora autorizada y permiten ejecutar las operaciones críticas sin conexión a Internet:

Contenedor	Tecnología	Responsabilidad
Aplicación Desktop	C# .NET 8, Avalonia UI, MVVM, Pharmacy.Domain y Pharmacy.Application	Interfaz de todos los roles. Ejecuta localmente ventas, caja y consultas de inventario mediante servicios de aplicación locales y consulta los servicios cloud cuando hay conexión

Contenedor	Tecnología	Responsabilidad
SQLite local	SQLite con EF Core (y Dapper para lecturas)	Única base del puesto, con tablas separadas lógicamente por módulo: productos, precios, lotes, stock local, ventas, detalle de ventas, movimientos de inventario, caja, Outbox y estado de sincronización
Sync Worker	C# BackgroundService dentro del proceso Desktop	Envía por lotes los mensajes pendientes del Outbox al Sync Service y aplica en SQLite los cambios descargados del servidor

4.3. Contenedores cloud

Los contenedores cloud se despliegan como imágenes Docker independientes sobre un VPS o proveedor cloud y comparten una misma instancia PostgreSQL administrada:

Contenedor	Tecnología	Responsabilidad
API Gateway	ASP.NET Core + YARP, Docker	Punto de entrada único para la Aplicación Desktop: enrutamiento, TLS, límites de solicitudes y registro de correlación
Sync Service	ASP.NET Core Web API, Docker	Fachada única de sincronización: recibe lotes del Outbox, valida idempotencia y versiones, los despacha al servicio de dominio correspondiente y entrega cambios incrementales
Identity Service	ASP.NET Core Identity + JWT, Docker	Usuarios, roles, permisos, dispositivos autorizados, emisión y renovación de tokens
Business Service	ASP.NET Core Web API, Pharmacy.Domain y Pharmacy.Application, Docker	Módulos de ventas, inventario, caja, compras, proveedores, clientes, recetas y reportes; agrupados por su fuerte necesidad transaccional
Notification Service	ASP.NET Core + BackgroundService, Docker	Alertas de vencimiento y stock mínimo, y envío de reportes programados por correo
Servidor de actualizaciones	Velopack, almacenamiento estático sobre HTTPS	Publica instaladores y paquetes delta que la Aplicación Desktop verifica e instala
PostgreSQL central	PostgreSQL administrado	Fuente consolidada central del negocio, organizada mediante schemas por dominio: sales, inventory, cash, identity, audit y sync

4.4. Almacenamiento local y central

El sistema utiliza dos niveles de almacenamiento con responsabilidades distintas. SQLite constituye la fuente operativa local del puesto: contiene únicamente la información necesaria para operar sin conexión y las operaciones reales realizadas mientras no hubo conectividad. PostgreSQL es la fuente consolidada central del negocio: reúne la información de todos los puestos y sucursales, la auditoría global, los reportes consolidados y las estadísticas, que no necesitan replicarse localmente.

En el diagrama existe un único contenedor PostgreSQL. Los schemas no se representan como bases de datos independientes, sino como una organización interna de la misma instancia:

Schema	Servicio responsable	Contenido principal
sales	Business Service	Ventas y detalle de ventas consolidados
inventory	Business Service	Productos, precios, lotes y movimientos de stock
cash	Business Service	Aperturas, movimientos, arqueos y cierres de caja
identity	Identity Service	Usuarios, roles, permisos, dispositivos y refresh tokens
audit	Business Service (escritura); consulta para reportes	Bitácora de operaciones críticas con usuario, dispositivo, fecha y datos de la operación
sync	Sync Service	Registro de eventos recibidos, claves de idempotencia, versiones por módulo y dispositivo

4.5. Protocolos y comunicación

Origen → Destino	Protocolo	Descripción
Aplicación Desktop → SQLite local	EF Core / Dapper sobre SQLite	Lectura y escritura transaccional de la operación diaria y del Outbox
Sync Worker → API Gateway → Sync Service	HTTPS / REST (JSON) + JWT	Envío de lotes del Outbox con clave de idempotencia y descarga incremental por módulo
Aplicación Desktop → API Gateway → Identity Service	HTTPS / REST (JSON)	Inicio de sesión, renovación de tokens y validación del dispositivo
Aplicación Desktop → API Gateway → Business Service	HTTPS / REST (JSON) + JWT	Consultas centrales en línea, como reportes consolidados y stock de otros puestos
Servicios cloud → Aplicación Desktop	WebSocket / SignalR (opcional)	Avisos en tiempo real, como cambios de precio o alertas de stock; no es imprescindible para operar
Sync Service → Business Service	HTTPS / REST interno	Despacho de eventos sincronizados al dominio correspondiente
Servicios cloud → PostgreSQL central	TCP 5432 con TLS (Npgsql / EF Core)	Cada servicio utiliza un usuario con permisos sobre su propio schema
Notification Service → Servicio de correo	SMTP con TLS	Envío de alertas y reportes programados
Aplicación Desktop → Servidor de actualizaciones	HTTPS	Consulta del canal de versiones y descarga de paquetes firmados

4.6. Operaciones con y sin conectividad

Cuando no existe conexión, el sistema distingue de forma explícita las operaciones que pueden ejecutarse en el puesto de aquellas que requieren los servicios cloud:

Tipo de operación	Operaciones
Disponibles sin conexión	Venta con pago en efectivo, emisión de recibo o factura simulada, apertura, movimientos, arqueo y cierre de caja, consulta de productos, precios, lotes y stock local, registro de movimientos de inventario
Requieren conectividad	Reportes consolidados y estadísticas globales, administración de usuarios y dispositivos, consulta del stock de otros puestos o sucursales
Diferidas hasta reconectar	Envío de las operaciones del Outbox al Sync Service, descarga de cambios de catálogo y precios, alertas por correo generadas en el servidor

Como todos los pagos se realizan en efectivo, el cobro se registra siempre en el puesto, con o sin conexión, y se consolida en el schema cash al sincronizar.

5. C3 – Diagrama de componentes: Aplicación Desktop

Este diagrama realiza un acercamiento al contenedor Aplicación Desktop. Su organización refleja que el puesto no es un cliente pasivo: contiene una capa de presentación, servicios de aplicación locales que ejecutan casos de uso reales, la lógica de dominio compartida con el servidor y una capa de persistencia y sincronización sobre la base SQLite local.

5.1. Diagrama de componentes de la Aplicación Desktop

Figura 3. Diagrama de componentes (C3) – Aplicación Desktop

Contenedor: Aplicación Desktop (C# .NET 8, Avalonia UI)

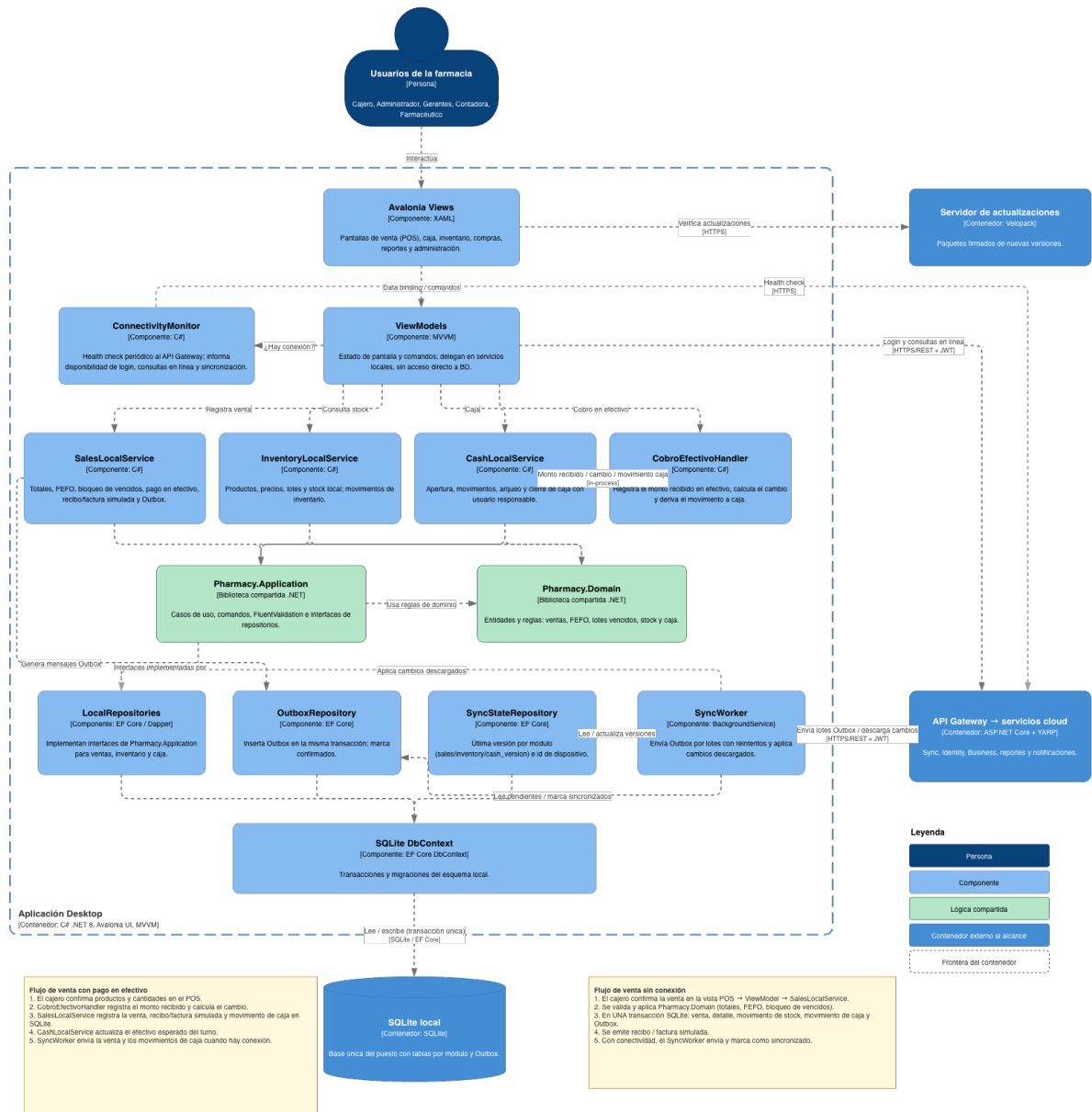


Figura 3. Diagrama de componentes (C3) de la Aplicación Desktop

5.2. Capa de presentación

La interfaz se construye con el patrón MVVM. Las Avalonia Views contienen únicamente la definición visual de las pantallas de venta, caja, inventario, compras, reportes y administración. Los ViewModels mantienen el estado de cada pantalla, exponen comandos y delegan toda operación de negocio en los servicios de aplicación locales, sin acceder directamente a la base de datos.

5.3. Servicios locales y lógica compartida

Componente	Responsabilidad
SalesLocalService	Registra ventas en el puesto: calcula totales, aplica FEFO, impide vender lotes vencidos, verifica la receta cuando el producto la requiere, registra el pago en efectivo, emite el recibo o factura simulada y genera los mensajes de Outbox correspondientes
InventoryLocalService	Consulta de productos, precios, lotes y stock local; registro de movimientos de inventario en el puesto
CashLocalService	Apertura, movimientos, arqueo y cierre de caja con registro del usuario responsable
Pharmacy.Domain	Reglas de dominio compartidas con el servidor: cálculo de ventas, FEFO, bloqueo de lotes vencidos, reglas de stock y de caja
Pharmacy.Application	Casos de uso, validaciones con FluentValidation e interfaces de repositorios que la capa local implementa sobre SQLite

5.4. Persistencia local y sincronización

Componente	Responsabilidad
LocalRepositories	Implementaciones sobre SQLite de las interfaces definidas en Pharmacy.Application para ventas, inventario y caja
SQLite DbContext	DbContext de Entity Framework Core sobre la única base SQLite del puesto; gestiona transacciones y migraciones del esquema local
OutboxRepository	Inserta los mensajes de Outbox dentro de la misma transacción de negocio y marca los mensajes confirmados por el servidor
SyncWorker	BackgroundService que envía los mensajes pendientes al Sync Service por lotes, con reintentos, y aplica los cambios descargados en SQLite
SyncStateRepository	Almacena la última versión conocida por módulo (sales_version, inventory_version, cash_version) y el identificador del dispositivo

5.5. Flujo de una venta sin conexión

El siguiente recorrido permite interpretar cómo colaboran los componentes del puesto en una venta realizada sin Internet:

- **Solicitud:** el cajero confirma la venta en la vista de punto de venta; el ViewModel envía el comando al SalesLocalService.
- **Reglas de dominio:** el servicio valida el comando y aplica las reglas de Pharmacy.Domain: cálculo de totales, selección de lotes por FEFO y bloqueo de lotes vencidos.
- **Transacción local:** en una única transacción SQLite se guardan la venta, el detalle de venta, el movimiento de stock, el movimiento de caja y el mensaje de Outbox; si alguna escritura falla, no se confirma ninguna.
- **Comprobante:** se emite el recibo o la factura simulada con numeración interna, sin validez fiscal.

- **Sincronización posterior:** cuando el SyncWorker detecta conectividad, envía el mensaje al Sync Service y lo marca como sincronizado al recibir la confirmación.

6. C3 – Diagrama de componentes: Backend Cloud

Este diagrama realiza un acercamiento al conjunto de servicios cloud. Todas las solicitudes externas ingresan por el API Gateway, que las dirige al servicio correspondiente. Los servicios se comunican con la instancia PostgreSQL central, y cada uno es responsable de su propio schema.

6.1. Diagrama de componentes del Backend Cloud

Figura 4. Diagrama de componentes (C3) – Backend Cloud
Servicios ASP.NET Core en Docker sobre PostgreSQL, administrador

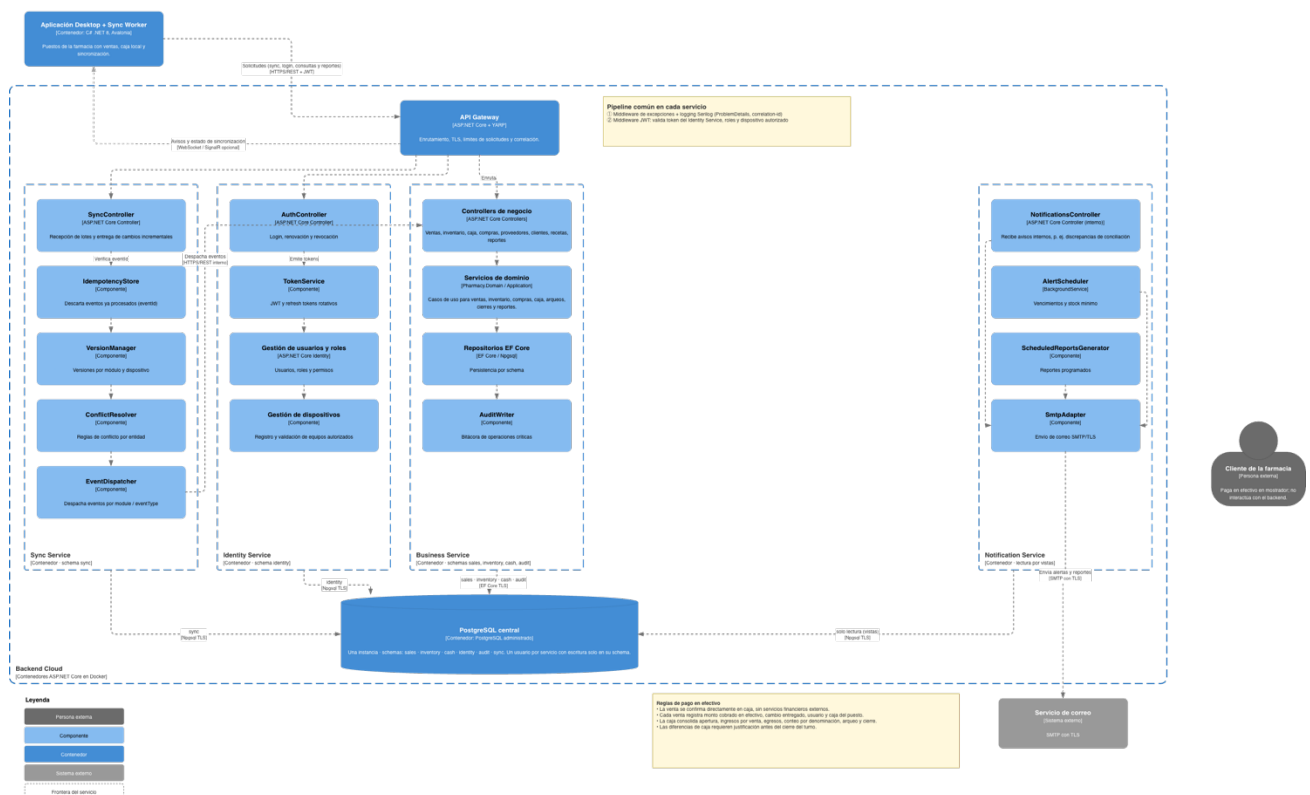


Figura 4. Diagrama de componentes (C3) del Backend Cloud

6.2. Punto de entrada y seguridad

El API Gateway concentra la entrada desde los puestos. En cada servicio, el pipeline de ASP.NET Core aplica primero el middleware de excepciones y logging con Serilog, que genera respuestas ProblemDetails y conserva el identificador de correlación, y luego el middleware JWT, que valida el token emitido por el Identity Service, los roles del usuario y el dispositivo autorizado.

6.3. Servicios cloud y sus componentes

Servicio	Componentes principales
Sync Service	SyncController (recepción de lotes y entrega de cambios), IdempotencyStore, VersionManager por módulo y dispositivo, EventDispatcher hacia el servicio de dominio y ConflictResolver
Identity Service	AuthController, servicio de tokens (JWT y refresh tokens rotativos), gestión de usuarios, roles, permisos y dispositivos autorizados con ASP.NET Core Identity
Business Service	Controllers y servicios de ventas, inventario, caja, compras, proveedores, clientes, recetas y reportes basados en Pharmacy.Domain y Pharmacy.Application; repositorios EF Core y escritor de auditoría
Notification Service	Programador de alertas (BackgroundService) para vencimientos y stock mínimo, generación de reportes programados y adaptador SMTP

6.4. Criterio de granularidad de los servicios

La arquitectura no divide cada tabla o CRUD en un microservicio. Se separan como servicios independientes únicamente los dominios con fronteras claras y ciclos de cambio propios: Identity, Sync y Notifications.

Ventas, inventario, caja y compras permanecen agrupados en el Business Service porque una venta modifica en la misma operación el stock y la caja, y separarlos obligaría a coordinar transacciones distribuidas. Dentro del servicio, cada módulo se organiza con su propio espacio de nombres y su propio schema, de modo que la arquitectura queda preparada para extraer otros dominios en el futuro si el volumen o el equipo lo justifican.

6.5. Acceso a PostgreSQL por schema

Todos los servicios utilizan la misma instancia PostgreSQL administrada, pero cada uno se conecta con un usuario propio que solo tiene permisos de escritura sobre su schema. Las consultas que requieren información de otros dominios, como los reportes consolidados, se realizan mediante vistas o permisos de solo lectura explícitamente concedidos. De este modo se obtiene separación de responsabilidades sin el costo de operar una base de datos física por servicio.

7. Sincronización local-cloud

La sincronización se mantiene centralizada. La Aplicación Desktop no sincroniza directamente con cada servicio cloud: el Sync Service es el único punto de entrada de sincronización para los equipos y es quien despacha cada evento al dominio que corresponde.

7.1. Flujo general

- **SQLite y Outbox:** cada operación local se guarda junto con su mensaje de Outbox en la misma transacción.
- **Sync Worker:** detecta conectividad y envía los mensajes pendientes por lotes, en orden cronológico, con reintentos y retroceso exponencial.
- **Sync Service:** registra cada evento en el schema sync, descarta los duplicados ya procesados y despacha los nuevos al Business Service según su módulo.
- **Servicio de dominio y PostgreSQL:** el Business Service aplica el evento con las reglas de Pharmacy.Domain y lo persiste en el schema correspondiente.
- **Confirmación:** el Sync Service responde con el estado de cada mensaje y el Sync Worker marca como sincronizados los confirmados.

7.2. Estructura del mensaje Outbox

Campo	Descripción
eventId	GUID único del evento; actúa como clave de idempotencia
deviceId	Identificador del puesto autorizado que generó la operación
module	Dominio al que pertenece el evento, por ejemplo sales, inventory o cash
eventType	Tipo de evento, por ejemplo SaleCreated o StockMovementCreated
entityId	GUID de la entidad afectada
version	Versión de la entidad utilizada para detectar conflictos
timestamp	Fecha y hora de la operación en el puesto
payload	Datos de la operación serializados en JSON

Por ejemplo, un mensaje con module = sales y eventType = SaleCreated es procesado por el módulo de ventas, mientras que un mensaje con module = inventory y eventType = StockMovementCreated corresponde al módulo de inventario.

7.3. Sincronización bidireccional y versiones

- **Local → Cloud:** envía las operaciones realizadas en el puesto, incluidas las registradas sin conexión.
- **Cloud → Local:** descarga productos, precios, lotes, cambios de inventario y demás información necesaria para que el puesto mantenga una copia operativa.

Cada módulo mantiene su propia versión de sincronización por dispositivo (sales_version, inventory_version, cash_version). El Sync Service devuelve únicamente los cambios posteriores a la última versión conocida por el puesto, lo que reduce el volumen de datos transferidos.

7.4. Idempotencia, conflictos y trazabilidad

El eventId permite que un mismo mensaje enviado varias veces se procese una sola vez. Los conflictos se resuelven según reglas definidas para cada entidad: las ventas y los movimientos se tratan como hechos que solo se agregan, el stock se recalcula a partir de los movimientos y los datos maestros, como precios y productos, prevalecen en su versión central. Cada evento conserva su usuario, dispositivo, fecha y versión, lo que mantiene la trazabilidad completa de las operaciones.

8. Registros de decisiones arquitectónicas (ADR)

Los registros de decisiones arquitectónicas documentan las decisiones más relevantes, su contexto, las alternativas evaluadas y sus consecuencias. Los ADR-001 a ADR-003 se mantienen y se actualizan para reflejar la nueva arquitectura; los ADR-004 a ADR-008 documentan las decisiones incorporadas en esta versión.

8.1. ADR-001: Framework del backend

Campo	Detalle
Identificador	ADR-001
Título	Selección de ASP.NET Core para los servicios cloud
Estado	Aceptado
Fecha	22 de septiembre de 2026
Contexto	El sistema necesita servicios cloud para la sincronización, la identidad, las notificaciones y el negocio. La aplicación de escritorio se desarrolla en C# con Avalonia UI, el equipo tiene experiencia en .NET y se requiere compartir reglas de negocio con el puesto, ejecutar procesos en segundo plano y desplegar en contenedores Docker.
Alternativas	A1. ASP.NET Core Web API (C#): alto rendimiento, Identity y JWT integrados, BackgroundService nativo y el mismo lenguaje que el cliente, lo que permite reutilizar Pharmacy.Domain y Pharmacy.Application. A2. Node.js con NestJS (TypeScript): ecosistema amplio, pero obliga a reescribir las reglas de negocio en otro lenguaje. A3. Spring Boot (Java): plataforma madura, con mayor consumo de recursos y sin experiencia previa del equipo. A4. Django REST Framework (Python): alta productividad en CRUD, con menor rendimiento y tipado débil para reglas complejas.

Campo	Detalle
Decisión	Se adopta ASP.NET Core (.NET 8 LTS) para todos los servicios cloud, cada uno desplegado como imagen Docker independiente.
Consecuencias positivas	Un solo lenguaje en todo el sistema, reutilización directa de la lógica compartida, pruebas homogéneas con xUnit y soporte nativo para Identity, JWT, SignalR y procesos en segundo plano.
Trade-offs y riesgos	El soporte de .NET 8 finaliza en noviembre de 2026, por lo que debe planificarse la migración a .NET 10 LTS. La elección concentra al equipo en el ecosistema .NET.

8.2. ADR-002: Estrategia de persistencia

Campo	Detalle
Identificador	ADR-002
Título	SQLite local por puesto y PostgreSQL administrado como fuente central
Estado	Aceptado
Fecha	22 de septiembre de 2026
Contexto	Las funciones críticas de venta y caja deben seguir operando sin Internet, mientras que la gerencia necesita información consolidada, auditoría y reportes de todos los puestos y futuras sucursales, con costos de infraestructura moderados.
Alternativas	A1. Solo base central en la nube: simple, pero impide operar sin conexión. A2. SQL Server local y central: robusto, pero con costo de licencias e instalación pesada en cada puesto. A3. Base documental central (MongoDB): flexible, pero sin transacciones relacionales adecuadas para ventas, caja e inventario. A4. SQLite local + PostgreSQL administrado central: operación local embebida y una base central relacional robusta y de código abierto.
Decisión	Se adopta la alternativa A4. SQLite es la fuente operativa local del puesto y PostgreSQL la fuente consolidada central del negocio; la organización interna de cada una se detalla en los ADR-005 y ADR-007.
Consecuencias positivas	Continuidad de la operación sin conexión, bajo costo de licencias, transacciones ACID en ambos extremos y respaldos automáticos en el servicio administrado.
Trade-offs y riesgos	Existen diferencias de tipos y funciones entre SQLite y PostgreSQL que deben cubrirse con pruebas de integración. El stock local de un puesto sin conexión puede diferir del stock central hasta la siguiente sincronización.

8.3. ADR-003: Mecanismo de sincronización

Campo	Detalle
Identificador	ADR-003
Título	Sincronización mediante Outbox Pattern y Sync Worker
Estado	Aceptado

Campo	Detalle
Fecha	22 de septiembre de 2026
Contexto	Las operaciones registradas localmente deben llegar al servidor sin pérdidas ni duplicados, y el puesto debe recibir los cambios centrales necesarios para seguir operando.
Alternativas	A1. Llamadas directas a la API en cada operación: simple, pero pierde datos cuando no hay conexión. A2. Cola de mensajería externa (p. ej. RabbitMQ): robusta, pero agrega infraestructura y no resuelve el almacenamiento local sin conexión. A3. Replicación nativa de base de datos: no es compatible entre SQLite y PostgreSQL y no aplica reglas de negocio. A4. Outbox local + Sync Worker + Sync Service: mensajes persistidos en la misma transacción y enviados cuando hay conectividad.
Decisión	Se adopta la alternativa A4, con mensajes identificados por eventId, deviceId, module, eventType, entityId, version, timestamp y payload, sincronización bidireccional y versiones por módulo.
Consecuencias positivas	Atomicidad local, idempotencia, envío por lotes con reintentos, descarga incremental y trazabilidad completa de cada evento.
Trade-offs y riesgos	Se requiere diseñar reglas de resolución de conflictos por entidad y monitorear los mensajes pendientes. Las operaciones realizadas sin conexión se consolidan con retraso.

8.4. ADR-004: Lógica de dominio compartida entre Desktop y servidor

Campo	Detalle
Identificador	ADR-004
Título	Bibliotecas Pharmacy.Domain y Pharmacy.Application compartidas
Estado	Aceptado
Fecha	22 de septiembre de 2026
Contexto	Las reglas de ventas, FEFO, stock y caja deben ejecutarse en el puesto durante la operación sin conexión y en el servidor al consolidar los eventos. Si se implementan por separado, pueden divergir y producir resultados distintos para una misma operación.
Alternativas	A1. Reglas solo en el servidor: el puesto no podría operar sin conexión. A2. Reglas duplicadas en el cliente y el servidor: permite operar sin conexión, pero con alto riesgo de inconsistencias y mayor mantenimiento. A3. Bibliotecas C# compartidas sin dependencias de infraestructura: una sola implementación reutilizada en ambos entornos.
Decisión	Se adopta la alternativa A3. Pharmacy.Domain y Pharmacy.Application no dependen de SQLite, PostgreSQL, HTTP, SMTP ni proveedores externos; cada entorno implementa sus interfaces.
Consecuencias positivas	Reglas únicas y consistentes, pruebas unitarias del dominio independientes de la infraestructura y menor esfuerzo de mantenimiento.
Trade-offs y riesgos	Toda versión del Desktop y del servidor debe ser compatible con la versión de las bibliotecas; un cambio de regla exige actualizar ambos entornos de forma coordinada mediante Velopack y el despliegue de los servicios.

8.5. ADR-005: PostgreSQL central con schemas por dominio

Campo	Detalle
Identificador	ADR-005
Título	Una instancia PostgreSQL con schemas separados por dominio
Estado	Aceptado
Fecha	22 de septiembre de 2026
Contexto	Los servicios cloud necesitan responsabilidades claras sobre sus datos, pero el proyecto debe mantener una infraestructura acotada y un costo moderado.
Alternativas	A1. Una base física por servicio: máximo aislamiento, pero multiplica la infraestructura, los respaldos y el costo. A2. Una sola base sin separación: simple, pero cualquier servicio puede modificar cualquier tabla. A3. Una instancia PostgreSQL con schemas por dominio y usuarios por servicio: separación lógica y de permisos sobre una misma infraestructura.
Decisión	Se adopta la alternativa A3 con los schemas sales, inventory, cash, identity, audit y sync. Cada servicio utiliza un usuario con permisos de escritura solo sobre su schema.
Consecuencias positivas	Responsabilidad clara sobre los datos, un único servicio administrado que respaldar y operar, y posibilidad de migrar un schema a otra base si en el futuro se extrae el dominio.
Trade-offs y riesgos	Todos los servicios comparten la disponibilidad y la capacidad de la misma instancia. Los reportes que cruzan dominios requieren vistas o permisos de lectura explícitos.

8.6. ADR-006: Servicios cloud separados

Campo	Detalle
Identificador	ADR-006
Título	Separación del backend en servicios con fronteras claras
Estado	Aceptado
Fecha	22 de septiembre de 2026
Contexto	La API única concentraba seguridad, sincronización, notificaciones y negocio en un mismo despliegue. Algunos de estos dominios tienen ciclos de cambio y requisitos de disponibilidad diferentes.
Alternativas	A1. Monolito único: simple de desplegar, pero cualquier cambio obliga a redespargar todo el backend. A2. Microservicio por entidad o CRUD: fronteras artificiales, transacciones distribuidas y exceso de infraestructura. A3. Servicios por frontera clara: Identity, Sync y Notifications separados, y un Business Service que agrupa ventas, inventario, caja y compras.
Decisión	Se adopta la alternativa A3, con un API Gateway como punto de entrada y una imagen Docker por servicio.

Campo	Detalle
Consecuencias positivas	Despliegue y escalamiento independientes de los servicios con fronteras claras, sin romper la consistencia transaccional entre venta, stock y caja; queda preparada la extracción futura de otros dominios.
Trade-offs y riesgos	Mayor número de despliegues, necesidad de trazas correlacionadas con Serilog y de comunicación interna entre servicios.

8.7. ADR-007: SQLite única por puesto

Campo	Detalle
Identificador	ADR-007
Título	Una única base SQLite local por computadora
Estado	Aceptado
Fecha	22 de septiembre de 2026
Contexto	Cada puesto necesita almacenar la información operativa de varios módulos y ejecutar transacciones que abarcan ventas, stock, caja y Outbox al mismo tiempo.
Alternativas	A1. Una base SQLite por módulo: aislamiento aparente, pero impide una transacción atómica entre venta, stock, caja y Outbox. A2. Una única base SQLite con tablas separadas lógicamente por módulo: transacción local atómica y administración simple.
Decisión	Se adopta la alternativa A2. La base contiene solo la información necesaria para operar sin conexión; los reportes consolidados, la auditoría global y las estadísticas permanecen en el servidor.
Consecuencias positivas	Atomicidad local de las operaciones, un solo archivo que respaldar y migrar, y menor volumen de datos replicados.
Trade-offs y riesgos	La base del puesto debe protegerse ante pérdida del equipo, y su esquema debe evolucionar mediante migraciones coordinadas con las actualizaciones de la aplicación.

8.8. ADR-008: Sync Service como fachada de sincronización

Campo	Detalle
Identificador	ADR-008
Título	Sync Service como punto único de sincronización de los puestos
Estado	Aceptado
Fecha	22 de septiembre de 2026
Contexto	Con varios servicios cloud, el Desktop podría sincronizar directamente con cada uno, lo que multiplicaría conexiones, credenciales y lógica de reintento en el cliente.

Campo	Detalle
Alternativas	A1. Sincronización directa con cada servicio: acopla el cliente a la estructura interna del backend. A2. Sync Service como fachada: un único punto de entrada que valida, registra y despacha los eventos al dominio correspondiente.
Decisión	Se adopta la alternativa A2. El Sync Service gestiona idempotencia, versiones por módulo y dispositivo y resolución de conflictos, y despacha cada evento según su module y eventType.
Consecuencias positivas	El cliente permanece simple y estable ante cambios internos del backend, y la sincronización se controla y audita en un solo lugar.
Trade-offs y riesgos	El Sync Service es un componente crítico y debe diseñarse para alta disponibilidad y monitoreo; su indisponibilidad retrasa la consolidación, aunque no detiene la operación local.

9. Relación con los requisitos y atributos de calidad

La siguiente tabla relaciona las áreas funcionales del SRS con los contenedores y componentes que las implementan en la nueva arquitectura:

Área	Contenedores	Componentes
Ventas	Aplicación Desktop, SQLite local, Business Service, PostgreSQL (sales)	SalesLocalService, Pharmacy.Domain, LocalRepositories, módulo de ventas del Business Service
Inventario	Aplicación Desktop, SQLite local, Business Service, PostgreSQL (inventory)	InventoryLocalService, reglas FEFO de Pharmacy.Domain, módulo de inventario del Business Service
Caja	Aplicación Desktop, SQLite local, Business Service, PostgreSQL (cash)	CashLocalService, Pharmacy.Domain, módulo de caja del Business Service
Operación sin conexión	Aplicación Desktop, SQLite local	Servicios locales, SQLite DbContext, OutboxRepository
Sincronización	Sync Worker, API Gateway, Sync Service, PostgreSQL (sync)	SyncWorker, SyncStateRepository, SyncController, IdempotencyStore, VersionManager, EventDispatcher
Pagos en efectivo	Aplicación Desktop, SQLite local, Business Service, PostgreSQL (cash)	SalesLocalService, CashLocalService, módulo de caja del Business Service
Usuarios y seguridad	API Gateway, Identity Service, PostgreSQL (identity)	AuthController, servicio de tokens, middleware JWT, gestión de dispositivos autorizados
Reportes	Business Service, Notification Service, PostgreSQL	Módulo de reportes, generación de reportes programados, adaptador SMTP
Auditoría	Business Service, Sync Service, PostgreSQL (audit)	Escritor de auditoría, registro de eventos con usuario, dispositivo y versión
Clientes y recetas	Aplicación Desktop, Business Service, PostgreSQL	SalesLocalService (verificación de receta en la venta), módulos de clientes y recetas del Business Service, Pharmacy.Application
Respaldo	PostgreSQL central administrado, SQLite local	Respaldos automáticos del servicio administrado; Outbox y SyncWorker como resguardo de las operaciones locales hasta su sincronización

Los atributos de calidad se atienden mediante las siguientes decisiones:

Atributo de calidad	Decisión arquitectónica
Disponibilidad	Ejecución local de ventas, caja e inventario sobre SQLite; la caída de Internet o de un servicio cloud no detiene la operación del puesto
Integridad	Transacción SQLite única por operación, Outbox en la misma transacción e idempotencia por eventId en el Sync Service
Consistencia	Reglas de negocio únicas en Pharmacy.Domain y Pharmacy.Application, ejecutadas en el puesto y en el servidor
Seguridad	JWT y refresh tokens, dispositivos autorizados, usuarios PostgreSQL con permisos por schema y TLS en todas las comunicaciones
Mantenibilidad	Servicios con fronteras claras, lógica compartida sin dependencias de infraestructura y pruebas con xUnit
Escalabilidad	Servicios desplegados por separado y schemas por dominio que permiten extraer dominios o bases en el futuro
Trazabilidad	Mensajes con usuario, dispositivo, fecha y versión, auditoría central y logging correlacionado con Serilog

10. Conclusiones

El modelo C4 permitió describir la arquitectura del Sistema de Gestión para Farmacéutica en niveles complementarios. El diagrama de contexto delimita la frontera del sistema y su única dependencia externa, el servicio de correo; el diagrama de contenedores distingue los contenedores locales, los contenedores cloud y los sistemas externos; y los diagramas de componentes detallan la organización interna de la Aplicación Desktop y del Backend Cloud.

La arquitectura resultante es local-first. La aplicación no es simplemente un cliente que consume una API, sino un cliente que ejecuta localmente un subconjunto de los casos de uso del negocio y posteriormente los sincroniza con la infraestructura central. PostgreSQL es la fuente consolidada central del negocio, mientras que SQLite constituye la fuente operativa local del puesto durante los períodos sin conectividad.

La lógica de dominio compartida evita duplicar las reglas entre el puesto y el servidor; el Sync Service, junto con el Outbox y el Sync Worker, garantiza una sincronización bidireccional, idempotente y trazable; y la separación en servicios con fronteras claras sobre una instancia PostgreSQL organizada por schemas equilibra la mantenibilidad con una infraestructura acotada.

Los registros ADR-001 a ADR-008 documentan estas decisiones con sus alternativas y trade-offs, y constituyen la referencia para mantener la coherencia de la arquitectura durante el desarrollo y la evolución del sistema hacia nuevas sucursales.